

open_ai.py

```
"""Driver that exercises each interview module (port of OpenAi.java). Every
module lives in its own file in this package:
- social_recommendations - friend-of-friend follow recommendations.
- plant_infection        - multi-source BFS ("Rotting Oranges").
- ip_addressing           - IPv4/IPv6 addressing toolkit.
- labeling_schedule       - human/model/task labeling schedules.
- chat_event_counter      - sliding 15-minute chat event window.
- gpu_credit_manager      - GPU credits with expiration.
- persistent_kv_store     - binary-safe persistent KV store.
- in_memory_table         - in-memory SQL-like single table.
- credit_ledger           - GPU credit ledger with out-of-order timestamps.
- map_serializer          - length-prefixed, binary-safe map serialization.
- sharded_kv_store        - persistent sharded KV store with replay restore.
- gpu_credit_ledger       - idempotent GPU credit ledger (non-negative
                           balances)."""
```

Output is Python-native (True/False, quoted strings in lists, dict/bytes repr), so it differs cosmetically from the Java driver's output (true/false, unquoted).
"""

```
from chat_event_counter import ChatEventCounter
from credit_ledger import CreditLedger
from gpu_credit_ledger import GpuCreditLedger
from gpu_credit_manager import GpuCreditManager
from in_memory_table import InMemoryTable
from ip_addressing import (
    parse_ipv4, adjacent_ipv4, cidr_range, ip_in_cidr, adjacent_ipv6,
)
from labeling_schedule import any_schedule, prefix_balanced_schedule
from map_serializer import solution as map_solution
from persistent_kv_store import PersistentKvStore
from plant_infection import infection_time
from sharded_kv_store import ShardedKvStore
from social_recommendations import SocialRecommendations

def main():
    # --- Module 1: Social Follow Recommendations ---
    ex1 = [
        ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
        ["follow", 2, 5], ["follow", 3, 6], ["recommend", 1, 3],
    ]
    print(SocialRecommendations.process_queries(ex1)) # [[4, 5, 6]]

    ex2 = [
        ["follow", 1, 1], ["follow", 1, 2], ["follow", 1, 2], ["follow", 2, 1],
        ["follow", 2, 3], ["recommend", 1, 5], ["unfollow", 1, 4],
        ["unfollow", 1, 2], ["recommend", 1, 5],
    ]
    print(SocialRecommendations.process_queries(ex2)) # [[3], []]

    # --- Module 2: Plant Infection Time (multi-source BFS) ---
    print(infection_time([[2, 1, 1], [1, 1, 0], [0, 1, 1]])) # 4
    print(infection_time([[2, 1, -1], [-1, 1, 1], [1, -1, 1]])) # -1
    # -> 0 (no healthy plants)
    print(infection_time([[0, 2, 0]]))

    # --- Module 3: IPv4 / IPv6 addressing toolkit ---
```

```

# Part 1: validate & parse IPv4
print(parse_ipv4("192.168.0.1")) # [192, 168, 0, 1]
print(parse_ipv4("0.0.0.0"))    # [0, 0, 0, 0]
# Part 2: adjacent IPv4
print(adjacent_ipv4("192.168.0.1", "up")) # 192.168.0.2
print(adjacent_ipv4("192.168.0.255", "up")) # 192.168.1.0
# Part 3: CIDR first & last
print(cidr_range("192.168.0.0/24")) # ['192.168.0.0', '192.168.0.255']
print(cidr_range("192.168.5.123/24")) # ['192.168.5.0', '192.168.5.255']
# Part 4: membership
print(ip_in_cidr("192.168.0.10", "192.168.0.0/24")) # 1
print(ip_in_cidr("192.168.0.200", "192.168.0.99/24")) # 1
# Part 5: adjacent IPv6 (period-separated)
print(adjacent_ipv6("ffff.ffff.ffff.ffff.ffff.ffff.ffff.ffff", "up"))
# ffff.ffff.ffff.ffff.ffff.ffff.ffff.ffff
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.ffff", "up"))
# 0000.0000.0000.0000.0000.0000.0000.0001

# --- Module 4: Labeling schedule generation ---
# Part 1: any valid schedule
print(any_schedule(3, 1, 2, 2))
# [['t1', 'm1', 'h1'], ['t2', 'm1', 'h1'], ['t2', 'm1', 'h2'],
#  ['t3', 'm1', 'h2']]
print(any_schedule(1, 1, 1, 1)) # [['t1', 'm1', 'h1']]
# Part 2: prefix-balanced schedule
print(prefix_balanced_schedule(3, 2, 4, 2))
# [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2'], ['t1', 'm1', 'h3'], ['t1', 'm2', 'h4'],
#  ['t2', 'm2', 'h1'], ['t2', 'm1', 'h2'], ['t2', 'm2', 'h3'], ['t2', 'm1', 'h4']]
print(prefix_balanced_schedule(1, 1, 1, 1)) # [['t1', 'm1', 'h1']]

# --- Module 5: Chat Event Counter (15-minute sliding window) ---
chat1 = [
    ["processEvent", "u1", "c1", 100], ["processEvent", "u1", "c1", 200],
    ["processEvent", "u1", "c1", 1000], ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat1)) # [3]
print(ChatEventCounter.simulate(["getCount", "u1", "c1"])) # [0]

# --- Module 6: GPU Credits with Expiration ---
gpu1 = [
    ["balance", 5], ["add", "a", 10, 5, 5], ["balance", 7],
    ["charge", 4, 8], ["balance", 8], ["balance", 10],
]
print(GpuCreditManager.simulate(gpu1)) # [0, 10, True, 6, 0]

gpu2 = [
    ["add", "a", 5, 10, 10], ["add", "b", 7, 5, 10], ["charge", 5, 12],
    ["balance", 12], ["balance", 17], ["charge", 6, 9], ["balance", 17],
]
print(GpuCreditManager.simulate(gpu2)) # [True, 7, 5, False, 5]

# --- Module 7: Persistent In-Memory KV Store ---
by = PersistentKvStore.by
# Part 1: serialize/deserialize a snapshot (str values)
kv1 = [
    ["put", "a", "1"], ["get", "a"], ["serialize", "snap1"],
    ["put", "a", "2"], ["deserialize", "snap1"], ["get", "a"],
]
# -> ['1', '1']

```

open_ai.py (continued)

```
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1)))
# Part 1: binary-safe with delimiter-like bytes in key/value
kv1b = [
    ["put", by("a|b"), by("v:1")], ["serialize", "p"],
    ["delete", by("a|b")],
    ["get", by("a|b")], ["deserialize", "p"], ["get", by("a|b")],
]
# -> [None, b'v:1']
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1b)))

# Part 2: snapshot split across size-bounded segments
kv2 = [
    ["put", by("a"), by("12345")], ["put", by("b"), by("67890")],
    ["serialize", "p"], ["segment_count", "p"], ["delete", by("a")],
    ["deserialize", "p"], ["get", by("a")], ["get", by("b")],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part2(15, kv2)))
# [6, b'12345', b'67890']
kv2b = [
    ["put", by("aa"), by("xx")], ["put", by("bb"), by("yy")],
    ["serialize", "snap"], ["segment_count", "snap"],
    ["reorder", "snap", [2, 0, 1]], ["delete", by("aa")],
    ["deserialize", "snap"], ["get", by("aa")], ["get", by("bb")],
]
print(PersistentKvStore.py_repr(
    PersistentKvStore.simulate_part2(20, kv2b)))
# [3, b'xx', b'yy']

# Part 3: snapshot + append-only log with replay and compaction
kv3 = [
    ["put", "a", "1"], ["put", "b", "2"], ["restart"],
    ["get", "a"], ["get", "b"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3)))
# ['1', '2', (2, 2, 0)]
kv3b = [
    ["put", "x", "1"], ["status"], ["put", "y", "2"], ["status"],
    ["restart"], ["get", "x"], ["get", "y"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(2, kv3b)))
# [(1, 1, 0), (2, 0, 1), '1', '2', (2, 0, 1)]

# --- Module 8: In-memory SQL-like table ---
sql1 = [
    "SET r1 name bob", "SET r1 age 2", "SET r2 name bob", "SET r2 age 10",
    "GET r1 name", "SELECT name bob age",
]
print(InMemoryTable.process_commands(sql1)) # ['bob', 'r2 r1']

sql2 = ["GET missing name", "SELECT status active score"]
print(InMemoryTable.process_commands(sql2)) # ['NULL', '']

# --- Module 9: Credit ledger with out-of-order timestamps ---
led1 = [
    ["GRANT", "alice", 10, 5], ["CHARGE", "alice", 4, 7],
    ["GET", "alice", 7],
    ["CHARGE", "alice", 10, 6], ["GET", "alice", 7],
]
print(CreditLedger.simulate(led1)) # [6, 0]
```

```

led2 = [
    ["GRANT", "alice", 5, 10], ["GRANT", "bob", 7, 1],
    ["CHARGE", "bob", 2, 3],
    ["GET", "bob", 2], ["GET", "bob", 3], ["GET", "alice", 9],
    ["GET", "alice", 10],
]
print(CreditLedger.simulate(led2)) # [7, 5, 0, 5]

# --- Module 10: Map serialization (length-prefixed, binary-safe) ---
print(map_solution("serialize", {})) # 0#
print(map_solution("deserialize", "0#")) # {}
sample_map = {"a": "hi", "b": ""}
encoded = map_solution("serialize", sample_map)
print(encoded) # 2#1#a2#hi1#b0#
print(map_solution("deserialize", encoded)) # {'a': 'hi', 'b': ''}

# --- Module 11: Persistent sharded KV store ---
shard1 = [
    ["put", "a", "1"], ["put", "b", "22"], ["get", "a"],
    ["put", "a", "333"],
    ["get", "a"], ["restore"], ["get", "b"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(10, shard1)))
# ([ '1', '333', '22'], [ 'P|a|1;', 'P|b|22;', 'P|a|333;'])
shard2 = [
    ["put", "ab", "x"], ["put", "c", "yz"], ["delete", "ab"],
    ["get", "ab"],
    ["restore"], ["get", "c"], ["get", "ab"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(20, shard2)))
# ([None, 'yz', None], [ 'P|ab|x;P|c|yz;D|ab;'])

# --- Module 12: Idempotent GPU credit ledger (non-negative balances) ---
led_init1 = [
    ["e1", "tenantA", 10], ["e2", "tenantA", -3], ["e3", "tenantB", 5],
]
led_ops1 = [
    ["get", "tenantA"], ["get", "tenantB"],
    ["apply", ["e4", "tenantA", 2]],
    ["apply", ["e5", "tenantB", -6]], ["get", "tenantB"],
    ["apply", ["e6", "tenantA", -4]], ["get", "tenantB"],
]
print(GpuCreditLedger.solution(led_init1, led_ops1))
# [7, 5, True, False, 5, True, 5]
led_ops2 = [
    ["get", "ghost"], ["apply", ["x1", "ghost", -3]], ["get", "ghost"],
    ["apply", ["x1", "ghost", -3]], ["apply", ["x2", "ghost", 4]],
    ["get", "ghost"],
]
print(GpuCreditLedger.solution([], led_ops2))
# [0, False, 0, False, True, 4]

if __name__ == "__main__":
    main()

```

chat_event_counter.py

```
"""In-memory counter for chat activity per exact (user_id, chat_id) pair.

process_event records one event;
get_count returns how many events for that
pair fall in the inclusive 15-minute (900 s) window ending at the pair's most
recent timestamp T, i.e. timestamps in [T - 900, T]. Returns 0 if the pair has
no events.
```

Because, for a given pair, process_event timestamps arrive non-decreasing, each pair's list is already sorted, so get_count is a binary search for the window start (no per-call sorting).

- process_event: $O(1)$ amortized append.
- get_count: $O(\log m)$ over that pair's m events.

```
"""
```

```
from bisect import bisect_left
```

```
# Approach (in plain terms):
# Keep a separate guest book (a list of timestamps) for each exact (user_id,
# chat_id) pair. Every event just stamps the current time on that pair's
# page, and because events for a pair arrive in time order the page stays
# sorted. To count the last 15 minutes (900 s): look at the newest stamp T,
# work out the cutoff T - 900, and since the page is already sorted, do a
# quick binary search to find where the cutoff lands. Everything from that
# spot to the end is inside the window, so the answer is how many stamps
# come after.
```

```
class ChatEventCounter:
```

```
    def __init__(self):
        # Time:  $O(1)$ , Space:  $O(1)$  - starts with an empty event dictionary.
        # (user_id, chat_id) -> ascending list of event timestamps.
        self._events = {}
```

```
@staticmethod
```

```
def _key(user_id, chat_id):
    # u = len(user_id), c = len(chat_id).
    # Time:  $O(u + c)$  - builds one key string from both ids.
    # Space:  $O(u + c)$  - the returned key string.
    # Length-prefix the user_id so distinct pairs can never collide
    # (e.g. ("a", "bc") vs ("ab", "c")).
    return f"{len(user_id)}:{user_id}{chat_id}"
```

```
def process_event(self, user_id, chat_id, timestamp):
    """Record one event for (user_id, chat_id) at the given timestamp."""
    # Time:  $O(1)$  amortized - one dictionary lookup plus a list append.
    # Space:  $O(1)$  - stores a single extra timestamp.
    key = self._key(user_id, chat_id)
    self._events.setdefault(key, []).append(timestamp)
```

```
def get_count(self, user_id, chat_id):
    """Events for (user_id, chat_id) within [T - 900, T], T = latest ts."""
    # m = number of events stored for this pair.
    # Time:  $O(\log m)$  - bisect_left binary-searches for the window start.
    # Space:  $O(1)$  - only a few index variables.
    ts = self._events.get(self._key(user_id, chat_id))
    if not ts:
        return 0
    latest = ts[-1] # list is sorted -> last is the max
    window_start = latest - 900
```

chat_event_counter.py (continued)

```
# bisect_left is a stdlib binary search:  $O(\log m)$  to find the first
# timestamp  $\geq$  window_start; everything from there to the end is inside
# the inclusive 900 s window.
lo = bisect_left(ts, window_start)
return len(ts) - lo

@staticmethod
def simulate(operations):
    """Replay operations, returning the result of every getCount in
    order."""
    # p = number of operations, m = events for the busiest pair.
    # Time:  $O(p \log m)$  - each getCount does one binary search.
    # Space:  $O(p)$  - the results list plus the stored timestamps.
    counter = ChatEventCounter()
    results = []
    for op in operations:
        kind = op[0]
        if kind == "processEvent":
            counter.process_event(op[1], op[2], op[3])
        elif kind == "getCount":
            results.append(counter.get_count(op[1], op[2]))
        else:
            raise ValueError(f"Unknown op: {kind}")
    return results

if __name__ == "__main__":
    # Test 1: three events for one pair, all inside the 15-minute window.
    chat1 = [
        ["processEvent", "u1", "c1", 100], ["processEvent", "u1", "c1", 200],
        ["processEvent", "u1", "c1", 1000], ["getCount", "u1", "c1"],
    ]
    print(ChatEventCounter.simulate(chat1)) # [3]

    # Test 2: asking about a pair that has no events at all -> 0.
    print(ChatEventCounter.simulate([["getCount", "u1", "c1"]])) # [0]

    # Test 3: a single event is always inside its own window -> 1.
    chat3 = [
        ["processEvent", "u1", "c1", 500], ["getCount", "u1", "c1"],
    ]
    print(ChatEventCounter.simulate(chat3)) # [1]

    # Test 4: event exactly 900 s before the latest -> boundary is inclusive.
    chat4 = [
        ["processEvent", "u1", "c1", 0], ["processEvent", "u1", "c1", 900],
        ["getCount", "u1", "c1"],
    ]
    print(ChatEventCounter.simulate(chat4)) # [2]

    # Test 5: event 901 s before the latest -> just outside the window.
    chat5 = [
        ["processEvent", "u1", "c1", 0], ["processEvent", "u1", "c1", 901],
        ["getCount", "u1", "c1"],
    ]
    print(ChatEventCounter.simulate(chat5)) # [1]

    # Test 6: only the newest event stays in the window; older ones drop off.
    chat6 = [
```

chat_event_counter.py (continued)

```
    ["processEvent", "u1", "c1", 0], ["processEvent", "u1", "c1", 500],
    ["processEvent", "u1", "c1", 2000], ["getCount", "u1", "c1"],
]
print(ChatEventCounter.simulate(chat6))                # [1]

# Test 7: two independent pairs are counted separately.
chat7 = [
    ["processEvent", "u1", "c1", 100], ["processEvent", "u1", "c1", 200],
    ["processEvent", "u2", "c2", 5000],
    ["getCount", "u1", "c1"], ["getCount", "u2", "c2"],
]
print(ChatEventCounter.simulate(chat7))                # [2, 1]

# Test 8: length-prefixed keys keep look-alike pairs from colliding.
chat8 = [
    ["processEvent", "a", "bc", 100], ["processEvent", "ab", "c", 200],
    ["getCount", "a", "bc"], ["getCount", "ab", "c"],
]
print(ChatEventCounter.simulate(chat8))                # [1, 1]
```

credit_ledger.py

"""A GPU credit ledger fed by a stream of requests whose timestamps are NOT necessarily increasing. Each GET reflects only the requests that have arrived so far, evaluated at the query timestamp; a later-arriving request with an earlier timestamp affects future GETs but never retroactively changes a GET that was already returned.

Request forms:

["GRANT", user, amount, timestamp]	add amount at timestamp
["CHARGE", user, amount, timestamp]	subtract amount IF the running balance at that moment is $\geq$ amount, else ignore
["GET", user, timestamp]	balance considering only arrived events with timestamp $\leq$ the query timestamp

Within a user, events are applied in (timestamp, arrival-order) order – a stable tie-breaker for equal timestamps. Each GET replays that user's eligible events from scratch so charge successes reflect the correct ordering.

```
class _Event:
    """One ledger event for a user."""

    def __init__(self, is_grant, amount, timestamp, arrival):
        # Time: O(1), Space: O(1) – just stores four fields.
        self.is_grant = is_grant
        self.amount = amount
        self.timestamp = timestamp
        # index in the input stream -> stable tie-breaker
        self.arrival = arrival

def _event_order(event):
    """Sort key: order events by (timestamp, arrival) so equal timestamps keep
    their original arrival order (a stable tie-break)."""
    # Time: O(1), Space: O(1) – returns a two-field tuple.
    return (event.timestamp, event.arrival)

# Approach (in plain terms):
#   Imagine each user has a shoebox where we drop in slips of paper as they
#   arrive: "+10 credits at time 5" or "-4 credits at time 7". Slips can be
#   dropped in any order and can even be back-dated.
#   When someone asks "what's the balance at time t?", we pull out only the
#   slips dated at or before t, lay them out in date order (using arrival order
#   to break ties), and add them up starting from zero. A "-" slip only counts
#   if there is enough money at that moment; otherwise we skip it. Because we
#   recompute from scratch every time, a slip that shows up late but is
#   back-dated changes future answers without
#   rewriting answers we already gave.
class CreditLedger:

    def __init__(self):
        # Time: O(1), Space: O(1) – starts with an empty per-user map.
        # user -> events that have arrived so far (in arrival order).
        self._ledger = {}

    def grant(self, user, amount, timestamp, arrival):
```



```

        """Record a GRANT for user."""
        # Time: O(1) amortized (dict lookup + list append),
        # Space: O(1) per call.
        self._ledger.setdefault(user, []).append(
            _Event(True, amount, timestamp, arrival))

def charge(self, user, amount, timestamp, arrival):
    """Record a CHARGE for user (success is decided later, at replay
    time)."""
    # Time: O(1) amortized (dict lookup + list append),
    # Space: O(1) per call.
    self._ledger.setdefault(user, []).append(
        _Event(False, amount, timestamp, arrival))

def get(self, user, timestamp):
    """Balance for user at timestamp, replaying eligible events in
    order."""
    # n = events stored for this user, k = eligible events (k <= n).
    # Time: O(n + k log k) – scan all n events, then sort the k
    #       eligible ones.
    # Space: O(k) – the list of eligible events.
    events = self._ledger.get(user)
    if events is None:
        return 0
    # Eligible events: timestamp <= query time.
    eligible = []
    for event in events:
        if event.timestamp <= timestamp:
            eligible.append(event)
    # Apply in (timestamp, arrival) order – stable tie-break by arrival.
    eligible.sort(key=_event_order)
    balance = 0
    for event in eligible:
        if event.is_grant:
            balance += event.amount
        elif balance >= event.amount:
            balance -= event.amount # charge applied only when affordable
    return balance

@staticmethod
def simulate(requests):
    """Process requests in arrival order, returning every GET in order."""
    # m = number of requests, n = events for the busiest user.
    # Time: O(m * n log n) worst case (each GET replays and sorts events).
    # Space: O(m + n) – the results list plus the stored events.
    ledger = CreditLedger()
    results = []
    for i, req in enumerate(requests):
        kind = req[0]
        user = req[1]
        if kind == "GRANT":
            ledger.grant(user, req[2], req[3], i)
        elif kind == "CHARGE":
            ledger.charge(user, req[2], req[3], i)
        elif kind == "GET":
            results.append(ledger.get(user, req[2]))
        else:
            raise ValueError(f"Unknown request: {kind}")
    return results

```

```
if __name__ == "__main__":
    # Test 1: a charge applied, then a back-dated charge drains the balance.
    led1 = [
        ["GRANT", "alice", 10, 5], ["CHARGE", "alice", 4, 7],
        ["GET", "alice", 7],
        ["CHARGE", "alice", 10, 6], ["GET", "alice", 7],
    ]
    print(CreditLedger.simulate(led1)) # [6, 0]

    # Test 2: two users kept independent; GET honors timestamp <= query time.
    led2 = [
        ["GRANT", "alice", 5, 10], ["GRANT", "bob", 7, 1],
        ["CHARGE", "bob", 2, 3],
        ["GET", "bob", 2], ["GET", "bob", 3],
        ["GET", "alice", 9], ["GET", "alice", 10],
    ]
    print(CreditLedger.simulate(led2)) # [7, 5, 0, 5]

    # Test 3: empty input -> no GETs -> empty result.
    led3 = []
    print(CreditLedger.simulate(led3)) # []

    # Test 4: GET for a user that never appears -> balance 0 (not found).
    led4 = [
        ["GET", "nobody", 5],
    ]
    print(CreditLedger.simulate(led4)) # [0]

    # Test 5: an unaffordable CHARGE is ignored, balance unchanged.
    led5 = [
        ["GRANT", "u", 3, 1], ["CHARGE", "u", 5, 2],
        ["GET", "u", 2], ["GET", "u", 5],
    ]
    print(CreditLedger.simulate(led5)) # [3, 3]

    # Test 6: a late, back-dated GRANT changes future GETs but not past
    # answers.
    led6 = [
        ["GRANT", "u", 10, 10], ["GET", "u", 10], ["GRANT", "u", 5, 3],
        ["GET", "u", 10], ["GET", "u", 5],
    ]
    print(CreditLedger.simulate(led6)) # [10, 15, 5]

    # Test 7: equal timestamps use arrival order - CHARGE before GRANT is
    # skipped.
    led7 = [
        ["CHARGE", "u", 5, 5], ["GRANT", "u", 10, 5], ["GET", "u", 5],
    ]
    print(CreditLedger.simulate(led7)) # [10]

    # Test 8: same timestamps, GRANT arrives first so the CHARGE succeeds.
    led8 = [
        ["GRANT", "u", 10, 5], ["CHARGE", "u", 5, 5], ["GET", "u", 5],
    ]
    print(CreditLedger.simulate(led8)) # [5]

    # Test 9: GET strictly before an event's timestamp excludes it (boundary).
```

credit_ledger.py (continued)

```
led9 = [  
    ["GRANT", "u", 8, 5], ["GET", "u", 4], ["GET", "u", 5],  
]  
print(CreditLedger.simulate(led9)) # [0, 8]
```

gpu_credit_ledger.py

```
"""In-memory credit ledger for a shared GPU pool with idempotent events.
```

```
Event schema (event_id, tenant_id, delta): a signed delta changes one tenant's balance. Rules:
```

- A balance may never go below 0; an event that would make it negative is rejected (no change).
- Idempotency: a repeated event_id does not change state again and returns the SAME accept/reject result produced the first time.
- Unknown tenants have balance 0.

```
Average O(1) per get/apply: a dict of balances plus a dict remembering each processed event_id's first result.
```

```
"""
```

```
# Approach (in plain terms):
```

- # Think of one prepaid wallet per tenant. Each event is a receipt with a unique ID saying "add or subtract this much for this tenant".
- # - Unknown tenants start at 0, and a wallet is never allowed to go negative: if a charge is too big we reject it and leave the wallet untouched.
- # - To stay idempotent we keep a notebook of every receipt ID we've already handled and the yes/no answer we gave. If the same ID shows up again we simply repeat that answer and change nothing - like ignoring a duplicate payment-confirmation email.

```
class GpuCreditLedger:
```

```
    def __init__(self):
```

```
        # Time: O(1), Space: O(1) - two empty dictionaries.
```

```
        self._balances = {}
```

```
        self._processed = {} # event_id -> first result
```

```
    def init(self, events):
```

```
        """Apply the initial events in order, using the same rules."""
```

```
        # n = number of initial events.
```

```
        # Time: O(n) average - one O(1)-average apply per event.
```

```
        # Space: O(n) - balances plus one remembered result per event_id.
```

```
        for event in events:
```

```
            self.apply(event[0], event[1], event[2])
```

```
    def get_balance(self, tenant_id):
```

```
        """Current balance for a tenant; 0 if unknown."""
```

```
        # Time: O(1) average - a single dict lookup. Space: O(1).
```

```
        return self._balances.get(tenant_id, 0)
```

```
    def apply(self, event_id, tenant_id, delta):
```

```
        """Apply an event idempotently. Returns True if accepted, False if rejected (would go negative). A duplicate event_id returns its original result without changing state."""
```

```
        # Time: O(1) average - a few dict lookups/inserts.
```

```
        # Space: O(1) amortized - may remember one balance and one result.
```

```
        prior = self._processed.get(event_id)
```

```
        if prior is not None:
```

```
            return prior # idempotent replay -> same result, no state change
```

```
        current = self._balances.get(tenant_id, 0)
```

```
        nxt = current + delta
```

```
        accepted = nxt >= 0
```

```
        if accepted:
```

```
            self._balances[tenant_id] = nxt
```

gpu_credit_ledger.py (continued)

```
self._processed[event_id] = accepted # remember the first-seen result
return accepted

@staticmethod
def solution(init_events, operations):
    """Apply init_events first, then run each op ("get" or "apply")."""
    # n = number of init events, m = number of operations.
    # Time: O(n + m) average – each event/op is O(1) average.
    # Space: O(n + m) – stored balances and remembered event results.
    ledger = GpuCreditLedger()
    ledger.init(init_events)
    results = []
    for op in operations:
        kind = op[0]
        if kind == "get":
            results.append(ledger.get_balance(op[1]))
        elif kind == "apply":
            event = op[1]
            results.append(ledger.apply(event[0], event[1], event[2]))
        else:
            raise ValueError(f"Unknown op: {kind}")
    return results

if __name__ == "__main__":
    # Test 1: init then mixed get/apply, including a rejected overdraft.
    init1 = [
        ["e1", "tenantA", 10], ["e2", "tenantA", -3], ["e3", "tenantB", 5],
    ]
    ops1 = [
        ["get", "tenantA"], ["get", "tenantB"],
        ["apply", ["e4", "tenantA", 2]],
        ["apply", ["e5", "tenantB", -6]], ["get", "tenantB"],
        ["apply", ["e6", "tenantA", -4]], ["get", "tenantB"],
    ]
    # -> [7, 5, True, False, 5, True, 5]
    print(GpuCreditLedger.solution(init1, ops1))

    # Test 2: unknown tenant starts at 0; duplicate event_id repeats first
    # result.
    ops2 = [
        ["get", "ghost"], ["apply", ["x1", "ghost", -3]], ["get", "ghost"],
        ["apply", ["x1", "ghost", -3]], ["apply", ["x2", "ghost", 4]],
        ["get", "ghost"],
    ]
    print(GpuCreditLedger.solution([], ops2)) # [0, False, 0, False, True, 4]

    # Test 3: no init events and no ops -> empty result.
    print(GpuCreditLedger.solution([], [])) # []

    # Test 4: a duplicate accepted event_id must NOT apply the delta twice.
    init4 = [["g1", "t", 5]]
    ops4 = [
        ["apply", ["g2", "t", 3]], ["get", "t"],
        ["apply", ["g2", "t", 3]], ["get", "t"],
    ]
    print(GpuCreditLedger.solution(init4, ops4)) # [True, 8, True, 8]

    # Test 5: reject an overdraft, then a smaller charge succeeds.
```

gpu_credit_ledger.py (continued)

```
init5 = [{"a", "t", 10}]
ops5 = [
    ["apply", ["b", "t", -15]], ["get", "t"],
    ["apply", ["c", "t", -4]], ["get", "t"],
]
print(GpuCreditLedger.solution(init5, ops5)) # [False, 10, True, 6]

# Test 6: zero delta is accepted; balance may land exactly on 0.
ops6 = [
    ["apply", ["z", "t", 0]], ["get", "t"], ["apply", ["p", "t", 5]],
    ["apply", ["m", "t", -5]], ["get", "t"],
]
print(GpuCreditLedger.solution([], ops6)) # [True, 0, True, True, 0]

# Test 7: a rejecting event inside init leaves the balance untouched.
init7 = [{"i1", "t", -5}, {"i2", "t", 8}]
print(GpuCreditLedger.solution(init7, [{"get", "t"}])) # [8]

# Test 8: idempotency is keyed by event_id globally, across tenants.
ops8 = [
    ["apply", ["dup", "tA", 5]], ["apply", ["dup", "tB", 5]],
    ["get", "tA"], ["get", "tB"],
]
print(GpuCreditLedger.solution([], ops8)) # [True, True, 5, 0]
```

gpu_credit_manager.py

```
"""Manage GPU credit batches that expire. A batch added as
(id, amount, timestamp, expiration) is usable during the half-open interval
[timestamp, timestamp + expiration).

- add(amount, timestamp, expiration): register a credit batch.
- charge(amount, timestamp): consume amount from batches active at that
  timestamp, soonest-to-expire first; all-or-nothing (no partial charge).
- balance(timestamp): total remaining credits active at that timestamp.

Operations are applied in input order; a later op may reference an earlier
timestamp but never retroactively changes an already-returned result.
expiration == 0 yields an empty interval (never active).
"""
```

```
class _Batch:
```

```
    """A credit batch with a mutable remaining amount and its active window."""
```

```
    def __init__(self, start, end, remaining):
        # Time: O(1), Space: O(1) - just stores three fields.
        self.start = start
        self.end = end          # exclusive: active in [start, end)
        self.remaining = remaining
```

```
    def active_at(self, t):
        # Time: O(1), Space: O(1) - two range comparisons.
        return self.start <= t < self.end and self.remaining > 0
```

```
def _batch_end(batch):
    """Sort key: a batch's expiration time (used to take
    soonest-expiring first)."""
    # Time: O(1), Space: O(1) - returns a single field.
    return batch.end
```

```
# Approach (in plain terms):
#   Think of each credit grant like a gift card that only works during a
#   certain time window and has some money left on it. We just keep a list
#   of all the gift cards.
#   - balance(t): add up the money left on every card that is still valid at
#     time t.
#   - charge(amount, t): look at all the cards valid at time t. If together
#     they don't have enough, don't charge anything (all-or-nothing). If they
#     do, spend from the card that expires soonest first, so credits get used
#     before they expire and go to waste.
#   Every request tells us its own time, so even if requests arrive out of
#   order, we simply check which cards are valid at that moment.
```

```
class GpuCreditManager:
```

```
    def __init__(self):
        # Time: O(1), Space: O(1) - starts with an empty batch list.
        self._batches = []
```

```
    def add(self, amount, timestamp, expiration):
        """Register a new credit batch active in
        [timestamp, timestamp + expiration)."""
        # Time: O(1) amortized (list append), Space: O(1) per call.
        self._batches.append(_Batch(timestamp, timestamp + expiration, amount))
```

```

def charge(self, amount, timestamp):
    """Consume amount from batches active at timestamp, soonest-expiring
    first. Returns True and applies the deduction only if the active
    balance covers the full amount; otherwise returns False and changes
    nothing."""
    # n = total batches, k = batches active at this timestamp (k <= n).
    # Time:  $O(n + k \log k)$  – scan all n batches, then sort the k
    #       active ones.
    # Space:  $O(k)$  – the list of active batches.
    # Collect the batches that are active at this timestamp.
    active = []
    total = 0
    for batch in self._batches:
        if batch.active_at(timestamp):
            active.append(batch)
            total += batch.remaining

    if total < amount:
        return False # insufficient active balance -> no change

    # Consume from the soonest-to-expire batches first.
    active.sort(key=_batch_end)
    need = amount
    for batch in active:
        if need <= 0:
            break
        take = min(batch.remaining, need)
        batch.remaining -= take
        need -= take
    return True

def balance(self, timestamp):
    """Total remaining credits usable at the given timestamp."""
    # Time:  $O(n)$  – scans all n batches once. Space:  $O(1)$ .
    total = 0
    for batch in self._batches:
        if batch.active_at(timestamp):
            total += batch.remaining
    return total

@staticmethod
def simulate(operations):
    """Simulate the manager, returning the result of every non-add op."""
    # m = number of operations, n = batches accumulated so far.
    # Time:  $O(m * n \log n)$  worst case (each charge sorts the active
    #       batches).
    # Space:  $O(m + n)$  – the results list plus the stored batches.
    mgr = GpuCreditManager()
    results = []
    for op in operations:
        kind = op[0]
        if kind == "add":
            # op[1] = id (unique metadata, unused by the logic)
            mgr.add(op[2], op[3], op[4])
        elif kind == "charge":
            results.append(mgr.charge(op[1], op[2]))
        elif kind == "balance":
            results.append(mgr.balance(op[1]))

```



```
        else:
            raise ValueError(f"Unknown op: {kind}")
        return results

if __name__ == "__main__":
    # Test 1: partial charge, then balances before and after expiry.
    gpu1 = [
        ["balance", 5], ["add", "a", 10, 5, 5], ["balance", 7],
        ["charge", 4, 8], ["balance", 8], ["balance", 10],
    ]
    print(GpuCreditManager.simulate(gpu1)) # [0, 10, True, 6, 0]

    # Test 2: two batches, soonest-to-expire consumed first.
    gpu2 = [
        ["add", "a", 5, 10, 10], ["add", "b", 7, 5, 10], ["charge", 5, 12],
        ["balance", 12], ["balance", 17], ["charge", 6, 9], ["balance", 17],
    ]
    print(GpuCreditManager.simulate(gpu2)) # [True, 7, 5, False, 5]

    # Test 3: no batches at all -> nothing to read or charge.
    gpu3 = [
        ["balance", 0], ["charge", 1, 0],
    ]
    print(GpuCreditManager.simulate(gpu3)) # [0, False]

    # Test 4: expiration == 0 -> empty interval, batch is never active.
    gpu4 = [
        ["add", "z", 100, 5, 0], ["balance", 5], ["charge", 1, 5],
    ]
    print(GpuCreditManager.simulate(gpu4)) # [0, False]

    # Test 5: window boundaries - start is inclusive, end is exclusive.
    gpu5 = [
        ["add", "a", 10, 5, 5], ["balance", 5], ["balance", 10],
        ["balance", 4], ["balance", 9],
    ]
    print(GpuCreditManager.simulate(gpu5)) # [10, 0, 0, 10]

    # Test 6: soonest-to-expire batch is drained before the later one.
    gpu6 = [
        ["add", "early", 5, 0, 10], ["add", "late", 5, 0, 20],
        ["charge", 5, 5], ["balance", 12], ["balance", 8],
    ]
    print(GpuCreditManager.simulate(gpu6)) # [True, 5, 5]

    # Test 7: all-or-nothing - an unaffordable charge changes nothing.
    gpu7 = [
        ["add", "a", 3, 0, 100], ["charge", 5, 10], ["balance", 10],
        ["charge", 3, 10], ["balance", 10],
    ]
    print(GpuCreditManager.simulate(gpu7)) # [False, 3, True, 0]

    # Test 8: several charges, including an out-of-order (earlier) timestamp.
    gpu8 = [
        ["add", "a", 10, 100, 50], ["charge", 4, 120], ["charge", 3, 110],
        ["balance", 130], ["charge", 5, 130], ["balance", 200],
    ]
    print(GpuCreditManager.simulate(gpu8)) # [True, True, 3, False, 0]
```


in_memory_table.py

```
"""A simple in-memory SQL-like database for a single table of string cells.
```

A row is identified by a string row_key; each row maps col_key -> value.

Commands (whitespace-separated tokens):

- SET row_key col_key value set/overwrite table[row_key][col_key].
- GET row_key col_key the value, or "NULL" if the cell is absent.
- SELECT where_col where_value order_by_col
 row_keys where table[row][where_col] == where_value, sorted by the
 value of order_by_col ascending (missing -> ""), ties broken by row_key
 ascending, joined by single spaces ("" if none match).

process_commands returns one output string per GET and SELECT, in order.

```
"""
```

```
def _select_sort_key(order_value_and_row_key):
```

```
    """Sort key for SELECT: the (order_by value, row_key) pair, both  
    ascending."""
```

```
    # Time: O(1), Space: O(1) - returns the decorated pair unchanged as  
    # the key.
```

```
    return order_value_and_row_key
```

```
# Approach (in plain terms):
```

```
# Picture a spreadsheet where every row has a name (row_key) and some  
# labelled boxes (columns) holding text. We store it as a dictionary of rows,  
# where each row is itself a small dictionary of column -> value.
```

```
# - SET writes into a box, creating the row if it's new or overwriting what  
# was already there.
```

```
# - GET reads one box, answering "NULL" when that row or box was never  
# filled.
```

```
# - SELECT means "give me every row whose column X equals Y", then line  
# those rows up sorted by another column's value (ties broken by the  
# row's name).
```

```
# To sort safely we first pair each matching row with the value we sort on,  
# sort those pairs, then read the names back out in order.
```

```
class InMemoryTable:
```

```
    def __init__(self):
```

```
        """Start with an empty table."""
```

```
        # Time: O(1), Space: O(1) - creates one empty dict.
```

```
        # row_key -> (col_key -> value); dict preserves insertion order.
```

```
        self._table = {}
```

```
    def set(self, row_key, col_key, value):
```

```
        """SET: set or overwrite a cell."""
```

```
        # Time: O(1) average, Space: O(1) - one dict insert (plus a new row
```

```
        # dict the first time a row_key is seen).
```

```
        self._table.setdefault(row_key, {})[col_key] = value
```

```
    def get(self, row_key, col_key):
```

```
        """GET: the cell value, or "NULL" if the row or column is absent."""
```

```
        # Time: O(1) average, Space: O(1) - at most two dict lookups.
```

```
        row = self._table.get(row_key)
```

```
        if row is None:
```

```
            return "NULL"
```

```
        value = row.get(col_key)
```

```
        return "NULL" if value is None else value
```

in_memory_table.py (continued)

```
def select(self, where_col, where_value, order_by_col):
    """SELECT: matching row_keys sorted by order_by_col then row_key."""
    # n = number of rows in the table, k = number of matching rows
    # (k <= n).
    # Time: O(n + k log k) – scan all n rows, then sort the k matches.
    # Space: O(k) – the list of decorated matches.
    # Pair every matching row with the value we will sort it by.
    decorated = []
    for row_key, row in self._table.items():
        if row.get(where_col) == where_value:
            sort_value = self._sort_value(row_key, order_by_col)
            decorated.append((sort_value, row_key))
    # Sort by order_by_col value, ties broken by row_key (both ascending).
    decorated.sort(key=_select_sort_key)
    # Read the row_keys back out, now in sorted order.
    matches = []
    for sort_value, row_key in decorated:
        matches.append(row_key)
    return " ".join(matches)

def _sort_value(self, row_key, order_by_col):
    """The order_by_col value for a row, or "" if the row lacks it."""
    # Time: O(1) average, Space: O(1) – one dict lookup.
    value = self._table[row_key].get(order_by_col)
    return "" if value is None else value

@staticmethod
def process_commands(commands):
    """Process commands, returning one output per GET/SELECT in order."""
    # m = number of commands, n = rows in the table.
    # Time: O(m * n log n) worst case (each SELECT scans and sorts the
    #       rows).
    # Space: O(m + n) – the outputs list plus the stored rows.
    db = InMemoryTable()
    outputs = []
    for command in commands:
        tokens = command.split(" ")
        if tokens[0] == "SET":
            db.set(tokens[1], tokens[2], tokens[3])
        elif tokens[0] == "GET":
            outputs.append(db.get(tokens[1], tokens[2]))
        elif tokens[0] == "SELECT":
            outputs.append(db.select(tokens[1], tokens[2], tokens[3]))
        else:
            raise ValueError(f"Unknown command: {tokens[0]}")
    return outputs

if __name__ == "__main__":
    # Test 1: GET a cell, then SELECT ordered by a numeric-looking string
    # column.
    sql1 = [
        "SET r1 name bob", "SET r1 age 2", "SET r2 name bob", "SET r2 age 10",
        "GET r1 name", "SELECT name bob age",
    ]
    print(InMemoryTable.process_commands(sql1)) # ['bob', 'r2 r1']

    # Test 2: GET a missing row -> NULL; SELECT with no matches -> "".
    print(InMemoryTable.process_commands(
```

in_memory_table.py (continued)

```
    ["GET missing name", "SELECT status active score"])) # ['NULL', '']

# Test 3: empty program -> no outputs at all.
print(InMemoryTable.process_commands([])) # []

# Test 4: overwrite -> the latest SET to a cell wins.
over = ["SET r1 v a", "SET r1 v b", "GET r1 v"]
print(InMemoryTable.process_commands(over)) # ['b']

# Test 5: GET a column the row never set -> NULL (row exists, cell does
# not).
absent = ["SET r1 name x", "GET r1 age"]
print(InMemoryTable.process_commands(absent)) # ['NULL']

# Test 6: SELECT whose filter matches nothing -> "".
no_match = ["SET r1 name x", "SELECT name y age"]
print(InMemoryTable.process_commands(no_match)) # ['']

# Test 7: ties on the order_by value are broken by row_key ascending.
ties = [
    "SET r3 name x", "SET r3 k 5", "SET r1 name x", "SET r1 k 5",
    "SET r2 name x", "SET r2 k 5", "SELECT name x k",
]
print(InMemoryTable.process_commands(ties)) # ['r1 r2 r3']

# Test 8: a missing order_by value sorts as "" (before any real value).
missing_order = ["SET a f m", "SET b f m", "SET b s 9", "SELECT f m s"]
print(InMemoryTable.process_commands(missing_order)) # ['a b']

# Test 9: ordering is lexicographic on the string value, not numeric.
numeric = [
    "SET r1 g h", "SET r1 n 9", "SET r2 g h", "SET r2 n 10",
    "SET r3 g h", "SET r3 n 100", "SELECT g h n",
]
print(InMemoryTable.process_commands(numeric)) # ['r2 r3 r1']
```

ip_addressing.py

```
"""Five self-contained IP exercises that share IPv4 parse/format helpers:
- parse_ipv4(ip)           validate + parse a dotted IPv4 into octets
- adjacent_ipv4(ip, dir)   the IPv4 one above/below
- cidr_range(cidr)         first + last address of a CIDR block
- ip_in_cidr(ip, cidr)     membership test for a CIDR block
- adjacent_ipv6(ip, dir)   the IPv6 one above/below (period-separated)
No networking libraries are used - only manual parsing and arithmetic.
"""
```

```
# Approach (in plain terms):
#   Treat an IP address like the number on a car's odometer. Each octet (or
#   hex group) is one "wheel", and the whole address is just one big number
#   written in separate pieces.
#   - parse: read the dotted text and make sure every wheel holds a legal
#   number.
#   - adjacent: roll the odometer one click up or down; when a wheel passes
#   its max it rolls over to 0 and nudges the wheel on its left (a carry),
#   and the mirror image happens when rolling down (a borrow).
#   - CIDR block: a "street" where every address shares the same leading
#   bits; the prefix says how many leading bits are fixed, which pins down
#   the first and last address on that street.
#   - membership: check whether an address falls between that street's first
#   and last address.
#   It is all plain integer and bit-mask math - no networking libraries.
```

```
# ----- Part 1: validate & parse IPv4 -----
def parse_ipv4(ip):
    """Parse a dotted-decimal IPv4 into 4 octets, or [] if invalid."""
    # n = length of the ip string.
    # Time: O(n) - scans each character once while validating.
    # Space: O(1) - 4 octets.
    octets = _to_octets(ip)
    if octets is None:
        return [] # invalid -> empty list
    return list(octets)
```

```
# ----- Part 2: adjacent IPv4 -----
def adjacent_ipv4(ip, direction):
    """The IPv4 one above ('up') or below ('down'); "" on
    invalid/over/underflow."""
    # n = length of the ip string.
    # Time: O(n) - one parse pass plus O(1) arithmetic. Space: O(1).
    octets = _to_octets(ip)
    if octets is None:
        return ""
    value = _octets_to_long(octets)
    if direction == "up":
        if value == 0xFFFFFFFF:
            return "" # would overflow above 255.255.255.255
        value += 1
    elif direction == "down":
        if value == 0:
            return "" # would underflow below 0.0.0.0
        value -= 1
    else:
        return "" # invalid direction
```

ip_addressing.py (continued)

```
    return _long_to_ipv4(value)

# ----- Part 3: first & last address of a CIDR block -----
def cidr_range(cidr):
    """Inclusive [first, last] addresses of A.B.C.D/P; [] if invalid."""
    # n = length of the cidr string.
    # Time: O(n) - parse the address and prefix once, then O(1) math.
    # Space: O(1).
    first_last = _cidr_first_last(cidr)
    if first_last is None:
        return []
    return [_long_to_ipv4(first_last[0]), _long_to_ipv4(first_last[1])]

# ----- Part 4: membership in a CIDR block -----
def ip_in_cidr(ip, cidr):
    """1 if ip is inside cidr (inclusive), 0 if not, -1 if either input is
    invalid."""
    # n = combined length of the ip and cidr strings.
    # Time: O(n) - parse both inputs once, then O(1) comparisons. Space: O(1).
    octets = _to_octets(ip)
    first_last = _cidr_first_last(cidr)
    if octets is None or first_last is None:
        return -1
    value = _octets_to_long(octets)
    return 1 if first_last[0] <= value <= first_last[1] else 0

# ----- Part 5: adjacent IPv6 (8 period-separated hex groups) -----
def adjacent_ipv6(ip, direction):
    """The IPv6 one above ('up') or below ('down'), normalized to eight
    lowercase 4-digit hex groups separated by '.'; "" on invalid input, invalid
    direction, or 128-bit over/underflow. Compressed '::' is NOT allowed."""
    # n = number of hex groups (always 8 when the input is valid).
    # Time: O(n) - parse the groups, propagate one carry/borrow, then
    #         format each group.
    # Space: O(n) - the list of 8 groups plus the hex pieces we join.
    groups = _to_groups(ip)
    if groups is None:
        return ""
    if direction == "up":
        up = True
    elif direction == "down":
        up = False
    else:
        return "" # invalid direction

    idx = 7
    if up:
        # propagate carry from the least-significant group
        while idx >= 0 and groups[idx] == 0xFFFF:
            groups[idx] = 0
            idx -= 1
        if idx < 0:
            return "" # overflow above ffff.ffff...ffff
        groups[idx] += 1
    else:
        # propagate borrow from the least-significant group
```

ip_addressing.py (continued)

```
while idx >= 0 and groups[idx] == 0:
    groups[idx] = 0xFFFF
    idx -= 1
if idx < 0:
    return "" # underflow below 0000.0000...0000
groups[idx] -= 1

# Build the eight hex pieces one at a time, then join them with '.'.
pieces = []
for group in groups:
    pieces.append(_to_hex4(group))
return ".".join(pieces)

# ----- shared helpers -----

def _to_octets(ip):
    """Validate and split a dotted-decimal IPv4 into 4 octets, or None if
    invalid."""
    # n = length of the ip string.
    # Time: O(n) - split, then validate every character once.
    # Space: O(1) - 4 octets.
    parts = _split_exact(ip, ".", 4)
    if parts is None:
        return None
    octets = []
    for part in parts:
        value = _parse_octet(part)
        if value < 0:
            return None
        octets.append(value)
    return octets

def _parse_octet(s):
    """Parse a decimal octet (digits only, leading zeros allowed, 0..255);
    -1 if invalid."""
    # n = number of characters in s (at most a handful).
    # Time: O(n) - one pass over the digits. Space: O(1).
    if not s:
        return -1
    value = 0
    for ch in s:
        if ch < "0" or ch > "9":
            return -1
        value = value * 10 + (ord(ch) - ord("0"))
    if value > 255:
        return -1 # also guards against overflow on long inputs
    return value

def _cidr_first_last(cidr):
    """[first, last] of a CIDR block, or None if invalid."""
    # n = length of the cidr string.
    # Time: O(n) - parse the address and prefix once, then O(1) bit math.
    # Space: O(1).
    parts = _split_exact(cidr, "/", 2)
    if parts is None:
        return None
```


ip_addressing.py (continued)

```
    octets = _to_octets(parts[0])
    if octets is None:
        return None
    prefix = _parse_prefix(parts[1])
    if prefix < 0:
        return None # prefix must be 0..32
    base = _octets_to_long(octets)
    mask = 0 if prefix == 0 else ((0xFFFFFFFF << (32 - prefix)) & 0xFFFFFFFF)
    first = base & mask
    last = first | (~mask & 0xFFFFFFFF)
    return [first, last]

def _parse_prefix(s):
    """Parse a CIDR prefix length (digits only); -1 if not an integer in
    [0, 32]."""
    # n = number of characters in s (at most a couple).
    # Time: O(n) - one pass over the digits. Space: O(1).
    if not s:
        return -1
    value = 0
    for ch in s:
        if ch < "0" or ch > "9":
            return -1
        value = value * 10 + (ord(ch) - ord("0"))
        if value > 32:
            return -1
    return value

def _to_groups(ip):
    """Parse the 8 period-separated hex groups (1..4 hex digits each); None
    if invalid."""
    # n = length of the ip string.
    # Time: O(n) - split, then validate every hex digit once.
    # Space: O(1) - 8 groups.
    parts = _split_exact(ip, ".", 8)
    if parts is None:
        return None
    groups = []
    for part in parts:
        value = _parse_hex_group(part)
        if value < 0:
            return None
        groups.append(value)
    return groups

def _parse_hex_group(s):
    """Parse a 1..4 digit hex group into 0..0xFFFF; -1 if invalid."""
    # n = number of characters in s (at most 4).
    # Time: O(n) - one pass over the hex digits. Space: O(1).
    if not s or len(s) > 4:
        return -1
    value = 0
    for ch in s:
        digit = _hex_val(ch)
        if digit < 0:
            return -1
```

ip_addressing.py (continued)

```
    value = value * 16 + digit
    return value

def _hex_val(c):
    # Time: O(1) - a few range checks on a single character. Space: O(1).
    if "0" <= c <= "9":
        return ord(c) - ord("0")
    if "a" <= c <= "f":
        return ord(c) - ord("a") + 10
    if "A" <= c <= "F":
        return ord(c) - ord("A") + 10
    return -1

def _to_hex4(v):
    """Format a 16-bit value as exactly 4 lowercase hex digits."""
    # Time: O(1) - formats a fixed-width 4-digit string. Space: O(1).
    return f"{v:04x}"

def _split_exact(s, delim, expected):
    """Split s on delim (no merging, trailing empties kept); return the parts
    only if there are exactly 'expected' of them, else None."""
    # n = length of the string s.
    # Time: O(n) - a single split pass. Space: O(n) - the list of split parts.
    if s is None:
        return None
    parts = s.split(delim)
    if len(parts) != expected:
        return None
    return parts

def _octets_to_long(octets):
    # Time: O(1) - four shifts/ors on a fixed-size list. Space: O(1).
    return (octets[0] << 24) | (octets[1] << 16) | (octets[2] << 8) | octets[3]

def _long_to_ipv4(value):
    # Time: O(1) - four shifts/masks and one format. Space: O(1).
    octet1 = (value >> 24) & 0xFF
    octet2 = (value >> 16) & 0xFF
    octet3 = (value >> 8) & 0xFF
    octet4 = value & 0xFF
    return f"{octet1}.{octet2}.{octet3}.{octet4}"

if __name__ == "__main__":
    # ----- parse_ipv4 -----
    print(parse_ipv4("192.168.0.1"))          # [192, 168, 0, 1]
    print(parse_ipv4("0.0.0.0"))              # [0, 0, 0, 0]
    print(parse_ipv4("255.255.255.255"))      # [255, 255, 255, 255]
    # -> [10, 0, 0, 1] (leading zeros allowed)
    print(parse_ipv4("010.0.0.1"))
    print(parse_ipv4("256.1.1.1"))           # [] (octet > 255)
    print(parse_ipv4("1.2.3"))               # [] (too few octets)
    print(parse_ipv4("1.2.3.4.5"))           # [] (too many octets)
    print(parse_ipv4("1.2.3.a"))             # [] (non-digit)
```

ip_addressing.py (continued)

```
print(parse_ipv4("")) # [] (empty string)

# ----- adjacent_ipv4 -----
print(adjacent_ipv4("192.168.0.255", "up")) # 192.168.1.0
print(adjacent_ipv4("192.168.1.0", "down")) # 192.168.0.255
print(adjacent_ipv4("0.0.0.0", "up")) # 0.0.0.1
# -> "" (overflow -> empty line)
print(adjacent_ipv4("255.255.255.255", "up"))
# -> "" (underflow -> empty line)
print(adjacent_ipv4("0.0.0.0", "down"))
# -> "" (bad direction -> empty line)
print(adjacent_ipv4("192.168.0.1", "sideways"))
# -> "" (invalid ip -> empty line)
print(adjacent_ipv4("999.1.1.1", "up"))

# ----- cidr_range -----
print(cidr_range("192.168.0.0/24")) # ['192.168.0.0', '192.168.0.255']
print(cidr_range("0.0.0.0/0")) # ['0.0.0.0', '255.255.255.255']
print(cidr_range("192.168.1.130/25")) # ['192.168.1.128', '192.168.1.255']
print(cidr_range("10.0.0.5/32")) # ['10.0.0.5', '10.0.0.5']
print(cidr_range("1.2.3.4/33")) # [] (prefix > 32)
print(cidr_range("1.2.3.4")) # [] (no prefix)

# ----- ip_in_cidr -----
print(ip_in_cidr("192.168.0.10", "192.168.0.0/24")) # 1
print(ip_in_cidr("192.168.0.0", "192.168.0.0/24")) # 1 (first address)
print(ip_in_cidr("192.168.0.255", "192.168.0.0/24")) # 1 (last address)
print(ip_in_cidr("192.168.1.10", "192.168.0.0/24")) # 0 (outside block)
# -> 1 (whole address space)
print(ip_in_cidr("10.0.0.0", "0.0.0.0/0"))
print(ip_in_cidr("bad", "192.168.0.0/24")) # -1 (invalid ip)
print(ip_in_cidr("192.168.0.1", "192.168.0.0/33")) # -1 (invalid cidr)

# ----- adjacent_ipv6 -----
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.ffff", "up"))
# 0000.0000.0000.0000.0000.0000.0001.0000
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0001.0000", "down"))
# 0000.0000.0000.0000.0000.0000.0000.ffff
# -> "" (overflow)
print(adjacent_ipv6("ffff.ffff.ffff.ffff.ffff.ffff.ffff.ffff", "up"))
# -> "" (underflow)
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.0000", "down"))
# -> "" (bad direction)
print(adjacent_ipv6("0000.0000.0000.0000.0000.0000.0000.0001", "left"))
# -> "" (only 7 groups)
print(adjacent_ipv6("0.0.0.0.0.0.0", "up"))
```

labeling_schedule.py

```
"""Generate labeling schedules. An assignment is [task_id, model_id, human_id];
tasks are t1..t{total_task}, models m1..m{total_model}, humans
h1..h{total_human}.
```

```
Every human must appear in at least k assignments, the same human never
annotates the same task twice, and every ID must be valid. Returns an empty
list if no valid schedule exists.
```

```
- any_schedule          Part 1: any valid schedule.
- prefix_balanced_schedule Part 2: additionally prefix-balanced.
"""
```

```
# Approach (in plain terms):
```

```
# Picture running a workshop where every helper (human) must finish k
# different tasks, and you want to hand out the work in a tidy, repeatable
# way.
# - any_schedule: give each helper k tasks in a row, but start each helper
# on a different task by rotating the starting point by the helper's
# number - like dealing cards - so nobody repeats a task and the work
# spreads out. Model is always m1 because Part 1 only cares about a valid
# assignment.
# - prefix_balanced_schedule: instead of finishing one helper at a time, go
# round by round. In round r every helper does the same task t{r+1} at
# once, and we rotate which model each helper uses. Because a whole round
# finishes before the next starts, if you stop reading the list at ANY
# point everyone has done nearly the same amount - that is what
# "prefix-balanced" means.
# No schedule is possible when k distinct tasks cannot fit ( $k > \text{total\_task}$ );
# when k is 0 there is simply nothing to schedule, so an empty list is the
# answer.
```

```
def any_schedule(total_task, total_model, total_human, k):
    """Part 1: any valid schedule. Each human gets k distinct tasks; the
    reference rotates each human's starting task by the human index (human i
    takes tasks (i+0)..(i+k-1) mod total_task). Model is always m1."""
    # h = number of humans, k = required assignments per human.
    # Time:  $O(h * k)$  - emits exactly k rows for each of the h humans.
    # Space:  $O(h * k)$  - the returned schedule holds one row per assignment.
    result = []
    if k == 0:
        return result # no assignments required -> empty schedule is valid
    if k > total_task:
        return result # cannot give a human k distinct tasks -> impossible
    for human_index in range(total_human): # human-major order
        for task_offset in range(k): # the k distinct tasks for this human
            task_index = (human_index + task_offset) % total_task
            result.append([f"t{task_index + 1}", "m1", f"h{human_index + 1}"])
    return result
```

```
def prefix_balanced_schedule(total_task, total_model, total_human, k):
    """Part 2: a prefix-balanced schedule. Emitted in k rounds; round r assigns
    task t{r+1} to every human, with human i using model
    m{((i + r) % total_model) + 1}. Every prefix satisfies both balance
    rules."""
    # h = number of humans, k = number of rounds (assignments per human).
    # Time:  $O(h * k)$  - emits one row per human across k rounds.
    # Space:  $O(h * k)$  - the returned schedule holds one row per assignment.
    result = []
    if k == 0:
        return result
```

labeling_schedule.py (continued)

```
if k > total_task:
    return result
for round_index in range(k): # round r -> task t{r+1}
    for human_index in range(total_human): # human index within the round
        model_index = (human_index + round_index) % total_model
        result.append(
            [f"t{round_index + 1}", f"m{model_index + 1}",
             f"h{human_index + 1}"])
return result

if __name__ == "__main__":
    # ----- any_schedule -----
    print(any_schedule(3, 1, 2, 2))
    # [['t1', 'm1', 'h1'], ['t2', 'm1', 'h1'], ['t2', 'm1', 'h2'],
    #  ['t3', 'm1', 'h2']]
    print(any_schedule(3, 1, 1, 1)) # [['t1', 'm1', 'h1']]
    print(any_schedule(2, 1, 3, 2)) # rotation wraps around the 2 tasks
    # [['t1', 'm1', 'h1'], ['t2', 'm1', 'h1'], ['t2', 'm1', 'h2'],
    #  ['t1', 'm1', 'h2'], ['t1', 'm1', 'h3'], ['t2', 'm1', 'h3']]
    print(any_schedule(3, 1, 2, 0)) # [] (k == 0 -> nothing to schedule)
    print(any_schedule(2, 1, 2, 3)) # [] (k > total_task -> impossible)
    print(any_schedule(5, 1, 0, 2)) # [] (no humans)

    # ----- prefix_balanced_schedule -----
    print(prefix_balanced_schedule(3, 2, 4, 2))
    # [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2'], ['t1', 'm1', 'h3'],
    #  ['t1', 'm2', 'h4'], ['t2', 'm2', 'h1'], ['t2', 'm1', 'h2'],
    #  ['t2', 'm2', 'h3'], ['t2', 'm1', 'h4']]
    print(prefix_balanced_schedule(3, 2, 2, 1)) # single round, models m1/m2
    # [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2']]
    # model index rotates each round
    print(prefix_balanced_schedule(2, 3, 2, 2))
    # [['t1', 'm1', 'h1'], ['t1', 'm2', 'h2'], ['t2', 'm2', 'h1'],
    #  ['t2', 'm3', 'h2']]
    print(prefix_balanced_schedule(3, 2, 4, 0)) # [] (k == 0)
    print(prefix_balanced_schedule(2, 2, 2, 3)) # [] (k > total_task)
```

map_serializer.py

```
"""Invertible, binary-safe encoding for a dict[str, str].
```

Because keys/values may contain any character (including '#', ':', '|', '=', spaces), a delimiter-only format is unsafe. Instead each field is LENGTH-PREFIXED as <len>#<content>, and after reading a length we consume exactly that many characters – so the content may contain anything.

Format: <pair_count>#<key_len>#<key><value_len>#<value>... with pairs written in lexicographically sorted key order, so the output is deterministic.

Example: {"a": "hi", "b": ""} -> "2#1#a2#hi1#b0#"; the empty map -> "0#".

```
"""
```

```
# Approach (in plain terms):
```

```
# We need to flatten a dictionary into one string and later rebuild it
# exactly, even when keys or values contain tricky characters like '#', ':'
# or spaces. The trick is the one shipping labels use: write the LENGTH of
# each piece right before the piece itself. To read it back you first read
# the number, then grab exactly that many characters – so the content can be
# anything without ever confusing the reader about where a piece ends.
# - serialize: first write how many pairs there are, then for each key and
#   each value write "<length>#<text>". Keys are sorted so the same map
#   always produces the exact same string.
# - deserialize: read the pair count, then repeatedly read a length, take
#   that many characters, and rebuild each key and value.
# It is like tearing off exactly the number of squares of tape someone told
# you to take, instead of guessing where one piece stops and the next begins.
```

```
def serialize(mapping):
```

```
    """Serialize a map to the length-prefixed format (keys sorted
    lexicographically)."""
```

```
    # n = number of pairs, m = total characters across all keys and values.
```

```
    # Time: O(n log n + m) – sort the n keys, then copy m characters of
    #       content.
```

```
    # Space: O(n + m) – the collected pieces and the joined output string.
```

```
    parts = [f"{len(mapping)}#"]
```

```
    for key in sorted(mapping): # deterministic key order
```

```
        value = mapping[key]
```

```
        parts.append(f"{len(key)}#{key}")
```

```
        parts.append(f"{len(value)}#{value}")
```

```
    return "".join(parts)
```

```
def deserialize(data):
```

```
    """Reconstruct the original map from a serialized string."""
```

```
    # L = length of the serialized string.
```

```
    # Time: O(L) – one left-to-right scan, reading each character a constant
```

```
    #       number of times. Space: O(L) – the rebuilt keys, values, and map.
```

```
    mapping = {}
```

```
    pos = [0]
```

```
    count = _read_length(data, pos)
```

```
    for pair_number in range(count):
```

```
        key = _read_field(data, pos)
```

```
        value = _read_field(data, pos)
```

```
        mapping[key] = value
```

```
    return mapping
```

```
def _read_length(s, pos):
```

```
    """Read a run of digits terminated by '#', returning the int; advance
```

```

    past '#'.'''
    # d = number of digit characters in this length token.
    # Time: O(d) - walks the digits once. Space: O(1).
    start = pos[0]
    while pos[0] < len(s) and s[pos[0]] != "#":
        ch = s[pos[0]]
        if ch < "0" or ch > "9":
            raise ValueError(f"Malformed: expected digit at index {pos[0]}")
        pos[0] += 1
    if pos[0] >= len(s) or start == pos[0]:
        raise ValueError("Malformed: missing length or '#'")
    length = int(s[start:pos[0]])
    pos[0] += 1 # skip the '#'
    return length

def _read_field(s, pos):
    """Read a length-prefixed field: <len>#<exactly len chars>."""
    # f = length of this field's content (taken from its length prefix).
    # Time: O(f) - reads the prefix and slices f characters.
    # Space: O(f) - the slice.
    length = _read_length(s, pos)
    if pos[0] + length > len(s):
        raise ValueError("Malformed: field length exceeds input")
    field = s[pos[0]:pos[0] + length]
    pos[0] += length
    return field

def solution(operation, data):
    """Judge-style entry point: "serialize" a dict -> str, or "deserialize" a
    str -> dict."""
    # Time/Space: O(1) dispatch that delegates to serialize or deserialize, so
    # the cost is whichever of those runs (see their complexity notes above).
    if operation == "serialize":
        return serialize(data)
    if operation == "deserialize":
        return deserialize(data)
    raise ValueError(f"Unknown operation: {operation}")

if __name__ == "__main__":
    # ----- serialize -----
    print(solution("serialize", {})) # 0#
    encoded = solution("serialize", {"a": "hi", "b": ""})
    print(encoded) # 2#1#a2#hi1#b0#
    print(serialize({"key": "value"})) # 1#3#key5#value
    print(serialize({"": ""})) # 1#0#0#
    print(serialize({"a#b": "c#d", "x": "y"})) # 2#3#a#b3#c#d1#x1#y
    print(serialize({"n": "12#34"})) # 1#1#n5#12#34

    # ----- deserialize (round-trips) -----
    print(solution("deserialize", encoded)) # {'a': 'hi', 'b': ''}
    print(deserialize("0#")) # {}
    print(deserialize("1#0#0#")) # {'': ''}
    # -> {'a#b': 'c#d', 'x': 'y'}
    print(deserialize(serialize({"a#b": "c#d", "x": "y"})))
    print(deserialize(serialize({"n": "12#34"}))) # {'n': '12#34'}

```

```
# ----- malformed input raises ValueError -----
try:
    deserialize("")
except ValueError as err:
    print("error:", err)          # error: Malformed: missing length or '#'
try:
    deserialize("xyz")
except ValueError as err:
    print("error:", err) # error: Malformed: expected digit at index 0
try:
    deserialize("1#5#ab")
except ValueError as err:
    print("error:", err) # error: Malformed: field length exceeds input

# ----- unknown operation raises ValueError -----
try:
    solution("frobnicate", {})
except ValueError as err:
    print("error:", err)          # error: Unknown operation: frobnicate
```


persistent_kv_store.py

```
"""A persistent key-value store with a binary-safe serialization format.
```

Keys and values are Python str or bytes blobs. Persistence uses LENGTH-PREFIXED encoding (no delimiters), so keys/values may contain any byte (including '|', ':', newlines):

```
    field = [type:1][len:4 big-endian][raw bytes]
    map    = [count:4][field key][field value] x count
```

- Part 1 (simulate_part1): serialize/deserialize a snapshot to a label.
- Part 2 (simulate_part2): split a snapshot into size-bounded, order-independent segments (each segment = [total:4][index:4][chunk]).
- Part 3 (simulate_part3): snapshot + append-only log with replay on restart and threshold-based compaction.

A str value round-trips as str (UTF-8) and a bytes value as bytes, so the demo output distinguishes 'x' (str) from b'x' (bytes).

```
"""
```

```
# ----- binary-safe codec -----
```

```
def _write_int(out, v):
    """Append a 4-byte big-endian integer (length or count) to the buffer."""
    # Time: O(1), Space: O(1) - writes a fixed 4 bytes.
    out.extend(v.to_bytes(4, "big"))
```

```
def _write_field(out, field):
    """Append one length-prefixed field: [type:1][len:4][raw bytes]."""
    # n = number of bytes in the field.
    # Time: O(n) - encode (if str) and copy n bytes.
    # Space: O(n) - the encoded copy.
    is_str = isinstance(field, str)
    data = field.encode("utf-8") if is_str else field
    out.append(1 if is_str else 0)
    _write_int(out, len(data))
    out.extend(data)
```

```
def _encode_map(mapping):
    """Serialize a map to a binary-safe blob."""
    # n = number of entries, b = total bytes across all keys and values.
    # Time: O(n + b) - one field per key and value. Space: O(n + b) - the blob.
    out = bytearray()
    _write_int(out, len(mapping))
    for key, value in mapping.items():
        _write_field(out, key)
        _write_field(out, value)
    return bytes(out)
```

```
class _Reader:
    """Sequential reader over a blob produced by _encode_map."""
```

```
    def __init__(self, blob):
        # Time: O(1), Space: O(1) - stores the blob and a cursor position.
        self._b = blob
        self._i = 0
```

persistent_kv_store.py (continued)

```
def read_int(self):
    # Time: O(1), Space: O(1) - reads a fixed 4-byte integer.
    v = int.from_bytes(self._b[self._i:self._i + 4], "big")
    self._i += 4
    return v

def read_field(self):
    # n = length of this field. Time: O(n) - slice n bytes (plus a UTF-8
    # decode for str). Space: O(n) - the returned bytes/str.
    tag = self._b[self._i]
    self._i += 1
    length = self.read_int()
    data = bytes(self._b[self._i:self._i + length])
    self._i += length
    return data.decode("utf-8") if tag == 1 else data

def _decode_map(blob):
    """Rebuild a map from a blob produced by _encode_map."""
    # n = number of entries, b = total key/value bytes.
    # Time: O(n + b) - read every field once.
    # Space: O(n + b) - the rebuilt map.
    reader = _Reader(blob)
    count = reader.read_int()
    mapping = {}
    for _ in range(count):
        key = reader.read_field()
        value = reader.read_field()
        mapping[key] = value
    return mapping

def _read_int_at(b, off):
    """Read a 4-byte big-endian integer at a fixed offset in a blob."""
    # Time: O(1), Space: O(1) - reads 4 bytes.
    return int.from_bytes(b[off:off + 4], "big")

# ----- Part 2 segment helpers -----
_SEG_HEADER = 8 # [total:4][index:4]

def _segment_index(segment):
    """Sort key: a segment's stored index header (bytes 4..8)."""
    # Time: O(1), Space: O(1) - reads one 4-byte integer header.
    return _read_int_at(segment, 4)

def _split_into_segments(snap, max_segment_size):
    """Split a snapshot blob into size-bounded, index-stamped segments."""
    # s = len(snap), t = number of segments produced.
    # Time: O(s) - copy the snapshot into chunks. Space: O(s) - the segments
    # together hold the whole snapshot (plus t small headers).
    chunk = max_segment_size - _SEG_HEADER # >= 1 since max_segment_size >= 9
    total = max(1, (len(snap) + chunk - 1) // chunk)
    segs = []
    for idx in range(total):
        off = idx * chunk
        length = min(chunk, len(snap) - off)
```

persistent_kv_store.py (continued)

```
    seg = bytearray()
    _write_int(seg, total)
    _write_int(seg, idx)
    if length > 0:
        seg.extend(snap[off:off + length])
    segs.append(bytes(seg))
return segs

def _join_segments(segments):
    """Reassemble the snapshot from segments given in any order."""
    # t = number of segments, s = total payload bytes.
    # Time: O(t log t + s) – sort by index header, then concatenate.
    # Space: O(s).
    # Segments may be in any order; sort by their stored index header.
    ordered = sorted(segments, key=_segment_index)
    out = bytearray()
    for seg in ordered:
        out.extend(seg[_SEG_HEADER:])
    return bytes(out)

# Approach (in plain terms):
# We need to save a dictionary to "disk" and read it back, even when
# keys or values contain tricky bytes (spaces, '|', ':', newlines). The
# trick: never rely on separators. Instead, right before every chunk we
# write its exact length – like labelling a parcel "5 bytes follow". The
# reader then grabs exactly that many bytes, so nothing inside can ever be
# mistaken for a separator (binary-safe).
# - Part 1: write the whole map as one length-prefixed blob and
#   read it back.
# - Part 2: chop that blob into fixed-size parcels, each stamped
#   with its index, so they can be stored or shuffled and later
#   sorted back into the right order.
# - Part 3: keep a saved snapshot plus a running list (the log) of
#   recent changes. On restart, load the snapshot then re-apply the
#   log. When the log grows past a threshold, fold everything into a
#   fresh snapshot (compaction) and clear the log.
class PersistentKvStore:

    @staticmethod
    def by(s):
        """Demo helper: Python bytes from a str (ISO-8859-1, 1 char == 1
        byte)."""
        # n = len(s). Time: O(n) – encode n characters.
        # Space: O(n) – the bytes.
        return s.encode("latin-1")

    @staticmethod
    def py_repr(results):
        """Render a results list Python-style (str -> 'x', bytes -> b'x',
        None -> None, tuple -> (...))."""
        # n = total size of results.
        # Time: O(n), Space: O(n) – builds a repr string.
        return repr(results)

# ----- Part 1: serialize / deserialize a single snapshot -----
    @staticmethod
    def simulate_part1(operations):
```

```

"""Part 1: serialize/deserialize a whole snapshot to and from a
label."""
# m = number of ops, s = current store's serialized size,
# r = get-results.
# Time: O(m * s) worst case (serialize/deserialize copy the whole
# store).
# Space: O(s + r) - a snapshot blob plus the results list.
store = {}
disk = {} # fake disk: label -> blob
results = []
for op in operations:
    kind = op[0]
    if kind == "put":
        store[op[1]] = op[2]
    elif kind == "delete":
        store.pop(op[1], None)
    elif kind == "get":
        results.append(store.get(op[1])) # value or None
    elif kind == "serialize":
        disk[op[1]] = _encode_map(store)
    elif kind == "deserialize":
        store = _decode_map(disk[op[1]])
    else:
        raise ValueError(f"Unknown op: {kind}")
return results

# ----- Part 2: snapshot split across size-bounded segments -----
@staticmethod
def simulate_part2(max_segment_size, operations):
    """Part 2: serialize into size-bounded segments that survive
    reordering."""
    # m = number of ops, s = serialized store size,
    # t = segments per snapshot.
    # Time: O(m * (s + t log t)) worst case.
    # Space: O(s + r) - segments plus the r results.
    store = {}
    disk = {} # prefix -> list of segment blobs
    results = []
    for op in operations:
        kind = op[0]
        if kind == "put":
            store[op[1]] = op[2]
        elif kind == "delete":
            store.pop(op[1], None)
        elif kind == "get":
            results.append(store.get(op[1]))
        elif kind == "serialize":
            disk[op[1]] = _split_into_segments(
                _encode_map(store), max_segment_size
            )
        elif kind == "segment_count":
            results.append(len(disk[op[1]]))
        elif kind == "reorder":
            cur = disk[op[1]]
            order = op[2]
            # Physical shuffle; the index header preserves the true order.
            shuffled = []
            for idx in order:
                shuffled.append(cur[idx])

```

```

        disk[op[1]] = shuffled
    elif kind == "deserialize":
        store = _decode_map(_join_segments(disk[op[1]]))
    else:
        raise ValueError(f"Unknown op: {kind}")
    return results

# ----- Part 3: snapshot + append-only log with replay & compaction -----
@staticmethod
def simulate_part3(compact_threshold, operations):
    """Part 3: snapshot + append-only log, replayed on restart and
    folded in on compaction."""
    # m = number of ops, s = store size, L = pending log length,
    # r = results.
    # Time: O(m * s) worst case (compaction/restart re-encode or replay).
    # Space: O(s + L + r) - snapshot, log, and results.
    store = {}
    # Persisted artifacts (survive a restart): a snapshot blob + a
    # pending log.
    snapshot = [_encode_map(store)]
    log = [] # list of (is_put, key, val); val is None for delete
    compactions = [0]
    results = []

    def maybe_compact():
        # Time: O(s) when it compacts (re-encode the whole store),
        # else O(1).
        # Space: O(s) for the fresh snapshot when compaction happens.
        if len(log) >= compact_threshold:
            snapshot[0] = _encode_map(store) # fresh full snapshot
            log.clear() # clear the pending log
            compactions[0] += 1

    for op in operations:
        kind = op[0]
        if kind == "put":
            k, v = op[1], op[2]
            store[k] = v
            log.append((True, k, v))
            maybe_compact()
        elif kind == "delete":
            k = op[1]
            store.pop(k, None)
            log.append((False, k, None))
            maybe_compact()
        elif kind == "get":
            results.append(store.get(op[1]))
        elif kind == "restart":
            # Throw away memory; reload the snapshot, then replay the log.
            store = _decode_map(snapshot[0])
            for is_put, k, v in log:
                if is_put:
                    store[k] = v
                else:
                    store.pop(k, None)
        elif kind == "status":
            # (live keys, pending log records, snapshots written)
            results.append((len(store), len(log), compactions[0]))
        else:

```

persistent_kv_store.py (continued)

```
        raise ValueError(f"Unknown op: {kind}")
    return results

if __name__ == "__main__":
    by = PersistentKvStore.by

    # ----- Part 1: single-snapshot serialize / deserialize -----
    # Test 1: deserialize restores an older snapshot, undoing a later put.
    kv1 = [
        ["put", "a", "1"], ["get", "a"], ["serialize", "snap1"],
        ["put", "a", "2"], ["deserialize", "snap1"], ["get", "a"],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1)))
    # ['1', '1']

    # Test 2: get a missing key -> None; delete then get -> None.
    kv1b = [["get", "nope"], ["put", "a", "1"], ["delete", "a"], ["get", "a"]]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1b)))
    # [None, None]

    # Test 3: binary-safe - keys/values with '|', ':' and a newline round-trip.
    kv1c = [
        ["put", by("a|b"), by("v:1\n")], ["serialize", "s"],
        ["put", by("a|b"), by("zzz")], ["deserialize", "s"],
        ["get", by("a|b")],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1c)))
    # [b'v:1\n']

    # Test 4: a str value stays str and a bytes value stays bytes.
    kv1d = [
        ["put", "k", "x"], ["put", by("k2"), by("x")],
        ["get", "k"], ["get", by("k2")],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part1(kv1d)))
    # ['x', b'x']

    # ----- Part 2: snapshot split into reorderable segments -----
    # Test 5: count segments, then read values back after a shuffle-proof join.
    kv2 = [
        ["put", by("a"), by("12345")], ["put", by("b"), by("67890")],
        ["serialize", "p"], ["segment_count", "p"], ["delete", by("a")],
        ["deserialize", "p"], ["get", by("a")], ["get", by("b")],
    ]
    print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part2(15, kv2)))
    # [6, b'12345', b'67890']

    # Test 6: reversing the segment order still reassembles the snapshot
    # correctly.
    kv2b = [
        ["put", by("k1"), by("AAAA")], ["put", by("k2"), by("BBBB")],
        ["serialize", "p"], ["segment_count", "p"],
        ["reorder", "p", [2, 1, 0]],
        ["deserialize", "p"], ["get", by("k1")], ["get", by("k2")],
    ]
    print(PersistentKvStore.py_repr(
        PersistentKvStore.simulate_part2(20, kv2b)))
    # [3, b'AAAA', b'BBBB']
```

```
# Test 7: an empty store still serializes to exactly one segment.
kv2c = [
    ["serialize", "e"], ["segment_count", "e"],
    ["deserialize", "e"], ["get", by("x")],
]
print(PersistentKvStore.py_repr(
    PersistentKvStore.simulate_part2(15, kv2c)))
# [1, None]

# ----- Part 3: snapshot + append-only log with replay & compaction -----
# Test 8: restart with no compaction yet -> empty snapshot + replayed log.
kv3 = [
    ["put", "a", "1"], ["put", "b", "2"], ["restart"],
    ["get", "a"], ["get", "b"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3)))
# ['1', '2', (2, 2, 0)]

# Test 9: hitting the threshold compacts, clearing the log and
# bumping the count.
kv3b = [
    ["put", "a", "1"], ["put", "b", "2"],
    ["put", "c", "3"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3b)))
# [(3, 0, 1)]

# Test 10: restart after a compaction reloads the snapshot then
# replays the log.
kv3c = [
    # compaction here
    ["put", "a", "1"], ["put", "b", "2"], ["put", "c", "3"],
    ["put", "d", "4"], ["delete", "a"], ["restart"],
    ["get", "a"], ["get", "d"], ["status"],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3c)))
# [None, '4', (3, 2, 1)]

# Test 11: binary-safe snapshot survives compaction + restart
# (mixed str/bytes).
kv3d = [
    ["put", by("k|1"), by("v:1")], ["put", "x", "y"], ["put", "p", "q"],
    ["restart"], ["get", by("k|1")],
]
print(PersistentKvStore.py_repr(PersistentKvStore.simulate_part3(3, kv3d)))
# [b'v:1']
```

plant_infection.py

```
"""Compute Plant Infection Time (multi-source BFS, a.k.a. "Rotting Oranges").

Grid values: 0 = empty ground, 1 = healthy plant, 2 = infected plant,
-1 = wall/blocked. Each minute every infected plant infects its orthogonally
adjacent healthy plants. Return the minutes until no plant can be infected, or
-1 if some healthy plant is unreachable, or 0 if there are no healthy plants to
begin with.
"""

from collections import deque

# Approach (in plain terms):
#   Picture every infected plant as a small fire that all start spreading at
#   the same moment. Each minute, every fire jumps to the healthy plants
#   directly next to it (up, down, left, right). We flood outward one ring at a
#   time and count the minutes as we go, stopping once no healthy plant can
#   catch fire. If every healthy plant was reached, the answer is the minutes
#   it took; if a plant is walled off and can never be reached, we return -1;
#   and if there were no healthy plants to begin with, it takes 0 minutes.
def infection_time(grid):
    """Minutes until every healthy plant is infected via multi-source BFS,
    -1 if some healthy plant is unreachable, or 0 if none need infecting."""
    # m = number of rows, n = number of columns.
    # Time: O(m * n) - each cell is enqueued and visited at most once.
    # Space: O(m * n) - the BFS queue can hold up to every cell.
    if not grid or not grid[0]:
        return 0 # no plants at all -> nothing to infect
    m, n = len(grid), len(grid[0])
    queue = deque()
    healthy = 0

    # Seed the BFS with every initially-infected plant; count healthy ones.
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                healthy += 1
    if healthy == 0:
        return 0 # nothing healthy -> 0 minutes

    dirs = ((1, 0), (-1, 0), (0, 1), (0, -1))
    minutes = 0

    # Expand the infection one layer (one minute) at a time.
    while queue and healthy > 0:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for dr, dc in dirs:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 1:
                    grid[nr][nc] = 2 # infect (also marks visited)
                    healthy -= 1
                    queue.append((nr, nc))
            minutes += 1

    # Any healthy plant left was unreachable.
    return minutes if healthy == 0 else -1
```


plant_infection.py (continued)

```
if __name__ == "__main__":
    # Test 1: classic spread from one corner reaches every plant in 4 minutes.
    print(infection_time([[2, 1, 1], [1, 1, 0], [0, 1, 1]]))    # 4

    # Test 2: walls trap some healthy plants -> unreachable, so -1.
    print(infection_time([[2, 1, -1], [-1, 1, 1], [1, -1, 1]])) # -1

    # Test 3: no healthy plants (only infected/empty) -> 0 minutes.
    print(infection_time([[0, 2, 0]]))                          # 0

    # Test 4: completely empty grid -> nothing to infect -> 0.
    print(infection_time([]))                                    # 0

    # Test 5: grid with only an empty row -> still nothing to infect -> 0.
    print(infection_time([[ ]]))                                # 0

    # Test 6: healthy plants but no infection source -> all unreachable -> -1.
    print(infection_time([[1, 1], [1, 1]]))                    # -1

    # Test 7: every plant already infected -> 0 minutes needed.
    print(infection_time([[2, 2], [2, 2]]))                    # 0

    # Test 8: a wall seals off the lone healthy plant -> -1.
    print(infection_time([[2, -1, 1]]))                        # -1

    # Test 9: single infected cell, nothing healthy -> 0.
    print(infection_time([[2]]))                                # 0

    # Test 10: single healthy cell with no source -> unreachable -> -1.
    print(infection_time([[1]]))                                # -1

    # Test 11: single empty cell -> 0.
    print(infection_time([[0]]))                                # 0
```

sharded_kv_store.py

```
"""Simulation of a persistent sharded key-value store. Each "shard file"
is just a string of appended records; every write is immediately
persisted to a shard.
```

```
Record encoding (keys/values are ASCII letters/digits, never '|' or ';'):
    put record:    P|key|value;
    delete record: D|key;
```

```
Sharding: append a record to the last shard unless that would make it longer
than shard_size; if it would exceed, start a new shard. (A single record is
guaranteed to fit in a fresh shard.)
```

```
Operations: put, get, delete, shutdown (no-op – already persisted), and
restore (discard the in-memory map and rebuild by replaying shards in
creation order, ignoring any incomplete trailing fragment not ending in
';').
"""
```

```
# Approach (in plain terms):
#   Imagine a notebook where you only ever ADD lines to the bottom, never
#   erasing. To remember key -> value we append "P|key|value;", and to
#   forget a key we append "D|key;" (a tombstone). When the current page
#   (shard) would overflow, we start a new page. For fast reads we also
#   keep the latest value of every key in a plain map. "Restore" is like
#   losing your memory and rebuilding it: re-read every page from the first
#   to the last and apply each line in order, so the last write wins and
#   deletes take effect. A half-written line at the very end (missing its
#   ';') is skipped.
```

```
class ShardedKvStore:
```

```
    def __init__(self, shard_size):
        """Start empty: remember the shard size limit, the map, and the
        shards."""
        # Time: O(1), Space: O(1) – stores the limit and two empty containers.
        self._shard_size = shard_size
        self._store = {}      # dict preserves insertion order
        self._shards = []    # each shard is a string of appended records
```

```
    def put(self, key, value):
        """put: persist a put record and set the current value."""
        # r = length of the new record, s = length of the last shard.
        # Time: O(r + s) – append to the last shard (string concat
        # rebuilds it).
        # Space: O(r + s) – the rebuilt shard string.
        self._append_record(f"P|{key}|{value};")
        self._store[key] = value
```

```
    def get(self, key):
        """get: the current value, or None if absent."""
        # Time: O(1) average, Space: O(1) – one dict lookup.
        return self._store.get(key)
```

```
    def delete(self, key):
        """delete: if the key exists, persist a tombstone and remove it."""
        # r = length of the tombstone, s = length of the last shard.
        # Time: O(r + s) when the key exists (append), else O(1).
        # Space: O(r + s).
        if key in self._store:
```

sharded_kv_store.py (continued)

```
        self._append_record(f"D|{key};")
        del self._store[key]

def shutdown(self):
    """shutdown: no-op in this simulation (every write is already
    persisted)."""
    # Time: O(1), Space: O(1) – does nothing; writes are already on "disk".

def restore(self):
    """restore: discard the in-memory map and rebuild it by replaying
    shards."""
    # S = total bytes across all shards,
    # k = number of live keys afterwards.
    # Time: O(S) – replay every record once. Space: O(k) – the rebuilt map.
    self._store.clear()
    for shard in self._shards:
        self._replay_shard(shard)

def _append_record(self, record):
    """Append to the last shard, or start a new shard if it would
    overflow."""
    # r = length of the record, s = length of the last shard.
    # Time: O(r + s) to concat onto a shard, or O(r) to start a fresh one.
    # Space: O(r + s) – the (possibly) rebuilt shard string.
    if (not self._shards
        or len(self._shards[-1]) + len(record) > self._shard_size):
        self._shards.append(record) # record always fits a fresh shard
    else:
        self._shards[-1] += record

def _replay_shard(self, shard):
    """Replay every complete ';'–terminated record; ignore a trailing
    fragment."""
    # L = length of the shard string.
    # Time: O(L) – walk the shard record by record. Space: O(1) extra.
    i = 0
    while i < len(shard):
        end = shard.find(";", i)
        if end < 0:
            break # incomplete trailing fragment -> ignore
        self._apply_record(shard[i:end]) # record body without the ';'
        i = end + 1

def _apply_record(self, rec):
    """Apply one decoded record body ("P|key|value" or "D|key") to the
    map."""
    # r = length of the record body.
    # Time: O(r) – slice out the key and value. Space: O(r) – the extracted
    # key/value strings.
    if rec.startswith("P|"):
        p1 = rec.index("|") # after 'P'
        p2 = rec.index("|", p1 + 1) # between key and value
        key = rec[p1 + 1:p2]
        value = rec[p2 + 1:]
        self._store[key] = value
    elif rec.startswith("D|"):
        self._store.pop(rec[2:], None)
    # any other (malformed) fragment is ignored
```

sharded_kv_store.py (continued)

```
def _shard_strings(self):
    """Final shard contents as plain strings."""
    # t = number of shards.
    # Time: O(t), Space: O(t) – a shallow copy of the shard list.
    return list(self._shards)

class Result:
    """Result of a run: the get outputs (in order) and the final
    shard list."""

    def __init__(self, get_results, shards):
        # Time: O(1), Space: O(1) – stores two references.
        self.get_results = get_results # entries may be None
        self.shards = shards

    @staticmethod
    def simulate(shard_size, operations):
        """Run a sequence of operations and return a Result(get_results,
        shards)."""
        # m = number of ops, S = total persisted bytes, g = number of gets.
        # Time: O(m + S) worst case (a restore replays everything).
        # Space: O(S + g) – the shards plus the collected get-results.
        db = ShardedKvStore(shard_size)
        get_results = []
        for op in operations:
            kind = op[0]
            if kind == "put":
                db.put(op[1], op[2])
            elif kind == "get":
                get_results.append(db.get(op[1]))
            elif kind == "delete":
                db.delete(op[1])
            elif kind == "shutdown":
                db.shutdown()
            elif kind == "restore":
                db.restore()
            else:
                raise ValueError(f"Unknown op: {kind}")
        return ShardedKvStore.Result(get_results, db._shard_strings())

    @staticmethod
    def py_repr(result):
        """Render a Result Python-tuple style: (['1', '333'], ['P|a|1;'])."""
        # n = total size of the result.
        # Time: O(n), Space: O(n) – a repr string.
        return repr((result.get_results, result.shards))

if __name__ == "__main__":
    # Test 1: overwrite persists a new record; restore rebuilds the
    # latest values.
    shard1 = [
        ["put", "a", "1"], ["put", "b", "22"], ["get", "a"],
        ["put", "a", "333"],
        ["get", "a"], ["restore"], ["get", "b"],
    ]
    print(ShardedKvStore.py_repr(ShardedKvStore.simulate(10, shard1)))
    # (['1', '333', '22'], ['P|a|1;', 'P|b|22;', 'P|a|333;'])
```

sharded_kv_store.py (continued)

```
# Test 2: delete writes a tombstone; the key stays gone after restore.
shard2 = [
    ["put", "ab", "x"], ["put", "c", "yz"], ["delete", "ab"],
    ["get", "ab"],
    ["restore"], ["get", "c"], ["get", "ab"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(20, shard2)))
# ([None, 'yz', None], ['P|ab|x;P|c|yz;D|ab;'])

# Test 3: empty program -> no gets and no shards.
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(10, [])))
# ([], [])

# Test 4: get a never-seen key -> None; deleting a missing key
# writes nothing.
shard4 = [
    ["put", "a", "1"], ["delete", "zzz"], ["get", "a"], ["get", "zzz"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(100, shard4)))
# (['1', None], ['P|a|1;'])

# Test 5: overwrite then restore keeps only the last value (both
# records kept).
shard5 = [
    ["put", "k", "v1"], ["put", "k", "v2"], ["restore"], ["get", "k"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(100, shard5)))
# (['v2'], ['P|k|v1;P|k|v2;'])

# Test 6: delete then restore -> replaying put then tombstone yields None.
shard6 = [
    ["put", "a", "1"], ["delete", "a"], ["restore"], ["get", "a"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(100, shard6)))
# ([None], ['P|a|1;D|a;'])

# Test 7: a record exactly filling a shard forces the next one onto
# a new shard.
shard7 = [
    ["put", "a", "1"], ["put", "b", "2"], ["get", "a"], ["get", "b"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(6, shard7)))
# (['1', '2'], ['P|a|1;', 'P|b|2;'])

# Test 8: many small shards; restore honors creation order (last
# write wins).
shard8 = [
    ["put", "x", "1"], ["put", "y", "2"], ["put", "x", "3"],
    ["delete", "y"],
    ["restore"], ["get", "x"], ["get", "y"],
]
print(ShardedKvStore.py_repr(ShardedKvStore.simulate(8, shard8)))
# (['3', None], ['P|x|1;', 'P|y|2;', 'P|x|3;', 'D|y;'])
```

social_recommendations.py

```
"""Social Follow Recommendations over an in-memory directed graph.
```

```
A -> B means user A follows user B. follow/unfollow mutate hash-based adjacency
sets in O(1) average time; recommend(user_id, k) uses a friend-of-friend rule,
ranking candidates by the number of distinct intermediate users that connect
user_id to the candidate.
```

```
"""
```

```
def _candidate_rank(item):
```

```
    """Sort key for a (candidate, intermediate_count) pair: more shared
    intermediates first (higher count), then smaller candidate id first."""
    # Time: O(1), Space: O(1) - unpacks a pair into a two-field tuple.
    candidate, count = item
    return (-count, candidate)
```

```
# Approach (in plain terms):
```

```
# Picture everyone's "following" list as a page in an address book naming who
# they follow. To suggest new people for someone, we use "friends of
# friends": look at each person they already follow, then look at who THOSE
# people follow. Every time one of your follows points at the same stranger,
# that stranger earns a point - more shared connections means a better
# suggestion. We skip yourself and anyone you already follow. Finally we hand
# back the top k strangers, most points first, breaking ties by the smaller
# id so the answer is always predictable.
```

```
class SocialRecommendations:
```

```
    def __init__(self):
```

```
        # Time: O(1), Space: O(1) - starts with an empty adjacency map.
        # adjacency set: user -> set of users they follow.
        self._following = {}
```

```
    def follow(self, follower_id, followee_id):
```

```
        """Add a directed edge follower -> followee. Self-follows and
        duplicates are ignored."""
        # Time: O(1) average - a dict lookup plus a set add.
        # Space: O(1) amortized - may store one new followee.
        if follower_id == followee_id:
            return # self-follow is ignored
        # setdefault + set.add naturally dedupes duplicate follows.
        self._following.setdefault(follower_id, set()).add(followee_id)
```

```
    def unfollow(self, follower_id, followee_id):
```

```
        """Remove the edge follower -> followee if present; otherwise a
        no-op."""
        # Time: O(1) average - a dict lookup plus a set discard. Space: O(1).
        followees = self._following.get(follower_id)
        if followees is not None:
            followees.discard(followee_id)
            if not followees:
                self._following.pop(follower_id, None) # keep the map tidy
```

```
    def recommend(self, user_id, k):
```

```
        """Up to k friend-of-friend recommendations for user_id.
```

```
A candidate is anyone followed by someone user_id already follows,
excluding user_id and anyone user_id already follows. Candidates are
ranked by the count of distinct intermediates (desc), ties broken by
```

```

    smaller user_id first."""
    # d = users user_id follows, E = total second-hop edges scanned,
    # c = number of distinct candidates found (c <= E).
    # Time: O(E + c log c) - scan every second-hop edge, then sort
    #       candidates.
    # Space: O(c) - the score map and the ranked list of candidates.
    result = []
    if k <= 0:
        return result
    direct = self._following.get(user_id)
    if not direct:
        return result # follows nobody -> no friend-of-friend candidates

    # Count how many distinct intermediates reach each candidate.
    score = {}
    for intermediate in direct:
        second_hop = self._following.get(intermediate)
        if second_hop is None:
            continue
        for candidate in second_hop:
            if candidate == user_id or candidate in direct:
                continue # exclude self and already-followed users
            score[candidate] = score.get(candidate, 0) + 1

    # Collect (candidate, count) pairs so a plain named key can rank them.
    ranked = []
    for candidate in score:
        ranked.append((candidate, score[candidate]))

    # Rank: higher score first, then smaller id first.
    ranked.sort(key=_candidate_rank)

    # Take the top k candidate ids, dropping the counts.
    limit = min(k, len(ranked))
    for index in range(limit):
        result.append(ranked[index][0])
    return result

@staticmethod
def process_queries(queries):
    """Process queries in order, returning the result of every
    recommend."""
    # q = number of queries; each recommend costs up to O(E + c log c)
    # (see recommend), while follow/unfollow are O(1) average.
    # Time: O(q * (E + c log c)) worst case.
    # Space: O(V + total_edges + q) - the stored graph plus the answers
    #       list.
    network = SocialRecommendations()
    answers = []
    for query in queries:
        op, a, b = query[0], query[1], query[2]
        if op == "follow":
            network.follow(a, b)
        elif op == "unfollow":
            network.unfollow(a, b)
        elif op == "recommend":
            answers.append(network.recommend(a, b))
        else:
            raise ValueError(f"Unknown op: {op}")

```

social_recommendations.py (continued)

```
    return answers

if __name__ == "__main__":
    # Test 1: user 1 -> {2,3}; 4 is reached by both 2 and 3 (score 2) so ranks
    # first.
    ex1 = [
        ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
        ["follow", 2, 5], ["follow", 3, 6], ["recommend", 1, 3],
    ]
    print(SocialRecommendations.process_queries(ex1)) # [[4, 5, 6]]

    # Test 2: self-follows/duplicates ignored; unfollow empties 1's set -> no
    # recs.
    ex2 = [
        ["follow", 1, 1], ["follow", 1, 2], ["follow", 1, 2], ["follow", 2, 1],
        ["follow", 2, 3], ["recommend", 1, 5], ["unfollow", 1, 4],
        ["unfollow", 1, 2], ["recommend", 1, 5],
    ]
    print(SocialRecommendations.process_queries(ex2)) # [[3], []]

    # Test 3: recommend for an unknown user who follows nobody -> empty list.
    ex3 = [
        ["recommend", 99, 3],
    ]
    print(SocialRecommendations.process_queries(ex3)) # [[]]

    # Test 4: k = 0 returns an empty list even when candidates exist.
    ex4 = [
        ["follow", 1, 2], ["follow", 2, 3], ["recommend", 1, 0],
    ]
    print(SocialRecommendations.process_queries(ex4)) # [[]]

    # Test 5: equal scores are tie-broken by the smaller candidate id.
    ex5 = [
        ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 9], ["follow", 3, 8],
        ["recommend", 1, 5],
    ]
    print(SocialRecommendations.process_queries(ex5)) # [[8, 9]]

    # Test 6: a candidate you already follow is excluded from recommendations.
    ex6 = [
        ["follow", 1, 2], ["follow", 1, 3],
        ["follow", 2, 3], ["recommend", 1, 5],
    ]
    print(SocialRecommendations.process_queries(ex6)) # [[]]

    # Test 7: k smaller than the candidate count returns only the top k.
    ex7 = [
        ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
        ["follow", 2, 5], ["follow", 3, 6], ["recommend", 1, 2],
    ]
    print(SocialRecommendations.process_queries(ex7)) # [[4, 5]]

    # Test 8: duplicate follows don't inflate a candidate's score.
    ex8 = [
        ["follow", 1, 2], ["follow", 1, 2],
        ["follow", 2, 5], ["recommend", 1, 5],
    ]
```


social_recommendations.py (continued)

```
print(SocialRecommendations.process_queries(ex8)) # [[5]]

# Test 9: unfollowing an intermediate removes the paths it provided.
ex9 = [
    ["follow", 1, 2], ["follow", 1, 3], ["follow", 2, 4], ["follow", 3, 4],
    ["unfollow", 1, 3], ["recommend", 1, 5],
]
print(SocialRecommendations.process_queries(ex9)) # [[4]]
```